IMPROVED BINARY SEARCH TREE USING C++

Data Structures and Algorithms — Project Report

Project focus: Tree creation, modification, searching, traversal, inspection, and memory cleanup

Project Type	Menu-driven C++ console application
Core Data Structure	Binary Search Tree (BST)
Implementation	Pointers, recursion, and a queue
Source File	main.cpp

Prepared as documentation for the improved BST implementation. Add student, class, roll number, and institution details here if required by your submission format.

Submitted for academic practical work

ABSTRACT

A Binary Search Tree (BST) is a linked, hierarchical data structure designed to maintain values according to an ordering rule. Each node stores one value and has at most two child pointers. Values smaller than the current node are stored in the left subtree, while values larger than the current node are stored in the right subtree. This arrangement makes it possible to search for values by comparing the target with each visited node rather than scanning every value in sequence.

This project presents a menu-driven BST application written in C++. The program retains the original operations—insertion, searching, deletion, inorder traversal, preorder traversal, and postorder traversal—and extends them with level-order traversal, minimum and maximum lookup, a readable parent-child structure display, safer handling of non-numeric input, explicit empty-tree messages, and cleanup of dynamically allocated nodes before exit.

The implementation uses a `Node` structure containing an integer and two pointers, recursive functions for most tree operations, and the C++ queue container for breadth-first traversal. Deletion is designed to handle all three structural cases: a leaf node, a node with one child, and a node with two children. In the two-child case, the inorder successor is used to preserve the BST ordering property.

The report explains the problem, objectives, data structure, program architecture, algorithms, sample test scenarios, complexity, limitations, and possible extensions. It is intended to make the code understandable for practical demonstration and oral examination. The included output document is a terminal-style rendering of a representative session; it is not claimed to be a photograph or screenshot captured from a real terminal.

REPORT ORGANIZATION

Section	Coverage
1–2	Introduction, problem statement, objectives, and scope
3–4	BST concepts and system design
5–6	Algorithms and implementation details
7–8	Testing, sample results, and complexity analysis
9–10	Limitations, future scope, and conclusion

1. INTRODUCTION

Data structures are essential for organizing information so that it can be accessed and updated efficiently. Arrays and linked lists are useful in many situations, but they do not inherently keep values in a hierarchical sorted arrangement. A tree represents relationships through nodes and edges. In a binary tree, each node has no more than two children, commonly called the left child and right child.

A Binary Search Tree adds an ordering condition to a binary tree. For any node, values in the left subtree are smaller than the node's value, and values in the right subtree are larger. This rule applies recursively to every subtree. Because of this structure, the program can choose which branch to follow during search, insertion, and deletion.

The project is implemented as a console application. When the program starts, the root pointer is set to `nullptr`, representing an empty tree. A menu then allows a user to select an operation. Each operation is handled by a dedicated function, which separates the tree logic from the user interface and makes the program easier to test and maintain.

1.1 Motivation

The purpose of this practical is to demonstrate pointer-based structures and recursive algorithms in a concrete program. Insertion builds the tree dynamically, search follows the ordering property, deletion updates links without violating the property, and traversal functions demonstrate different visiting orders. The added level-order traversal introduces a queue-based breadth-first algorithm.

1.2 Project Scope

The program manages integer values in memory during one execution. It does not save values to a file or database, and it does not balance the tree automatically. These boundaries keep the project focused on fundamental BST operations while leaving clear opportunities for future improvements.

1.3 Intended Users

The application is intended for students learning trees, recursion, pointers, dynamic memory, and algorithm analysis. The menu and textual output allow operations to be demonstrated step by step during a lab session or viva.

2. PROBLEM STATEMENT AND OBJECTIVES

2.1 Problem Statement

Develop a menu-driven C++ program that creates and maintains a Binary Search Tree of integer values. The program must allow insertion, search, deletion, inorder traversal, preorder traversal, and postorder traversal. To make the implementation more useful, it must also provide level-order traversal, minimum and maximum lookup, and a readable display of the parent-child structure. It should give understandable responses for invalid input, duplicate values, missing values, and operations attempted on an empty tree. All dynamically allocated nodes must be released before normal program termination.

2.2 Objectives

- Represent each tree node using an integer and left/right pointers.
- Insert values while preserving the BST ordering property.
- Search for a target by comparing it with the current node.
- Delete nodes correctly when they have zero, one, or two children.
- Implement inorder, preorder, and postorder depth-first traversals.
- Implement level-order traversal with a queue.
- Locate the smallest and largest values without traversing every node.
- Display the tree in an indented parent-child format.
- Handle non-numeric input without leaving the input stream in a failed state.
- Release all remaining nodes before the program exits.

2.3 Functional Requirements

ID	Requirement	Expected behavior
FR-01	Insert	Create a node at the correct position; ignore duplicates.
FR-02	Search	Report whether the target value exists.
FR-03	Delete	Handle leaf, one-child, and two-child nodes.
FR-04	Traversals	Display values in the requested order.
FR-05	Min/max	Return the leftmost/rightmost value.
FR-06	Input/exit	Reject invalid text and free nodes on exit.

3. BACKGROUND: BINARY SEARCH TREES

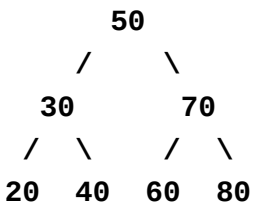
3.1 Node Representation

Each node contains three fields: `data`, `left`, and `right`. The data field stores the integer value. The left pointer references the root of the left subtree, and the right pointer references the root of the right subtree. If a child does not exist, its pointer is `nullptr`. The constructor initializes both child pointers to null so that newly created nodes have a known starting state.

3.2 The BST Ordering Property

For every node N, all values in N's left subtree are smaller than N's value and all values in N's right subtree are larger. The same rule must hold for every descendant. The insertion function ignores duplicate values, so this implementation stores each integer at most once.

3.3 Example Tree



In this example, 50 is the root. The values 30 and 70 are its left and right children. The values 20 and 40 are children of 30, while 60 and 80 are children of 70. Each subtree follows the same ordering rule.

3.4 Why Inorder Traversal Is Sorted

Inorder traversal visits the left subtree first, then the current node, and finally the right subtree. Since all values in the left subtree are smaller and all values in the right subtree are larger, this order produces values in ascending order. For the example tree, the output is `20 30 40 50 60 70 80`.

3.5 Balanced and Skewed Trees

A balanced tree has left and right subtrees of comparable height, so the search path is relatively short. A skewed tree resembles a linked list. If values are inserted in increasing order into this implementation, each new value may become the right child of the previous node. Operations then require up to O(n) time. The implementation does not perform rotations or automatic balancing.

4. SYSTEM DESIGN AND PROGRAM STRUCTURE

4.1 Overall Flow

The program begins by assigning `nullptr` to the root pointer. It displays the menu and reads the user's choice. The selected case requests any additional input, calls the relevant function, and prints a result. Unless the user selects Exit, control returns to the menu. On exit, the program recursively destroys the tree and then terminates.

4.2 Main Components

Component	Responsibility
Node structure	Stores data and child pointers; initializes pointers safely.
insertNode	Places a value according to BST ordering.
searchNode	Checks whether a value is present.
deleteNode	Removes a node and reconnects the affected subtree.
findMin / findMax	Follow left or right pointers to an extreme value.
Traversal functions	Visit nodes in depth-first or breadth-first order.
displayStructure	Prints node values with indentation and child labels.
destroyTree	Deletes nodes recursively before program exit.
readInt / showMenu	Separate input validation and menu presentation from tree logic.

4.3 Recursion and Base Cases

Most operations use recursion. A base case prevents the function from continuing when a null pointer is reached. For example, search returns false when the current node is null, and traversal returns immediately for a null subtree. These base cases are essential for correctness and prevent infinite recursive calls.

4.4 Queue for Level-order Traversal

Level-order traversal visits the root before its children and processes nodes level by level. A queue follows first-in, first-out order. The root is placed in the queue first; each iteration removes the front node, prints its value, and adds its existing children. This ensures that shallower nodes are processed before deeper nodes.

5. ALGORITHM AND IMPLEMENTATION DETAILS

5.1 Insertion

If the current pointer is null, insertion creates a new node and returns it. If the new value is smaller than the current node, insertion continues into the left subtree. If it is larger, insertion continues into the right subtree. Equal values are not inserted. The returned pointer is assigned back to the corresponding child link, ensuring that newly created nodes become part of the tree.

5.2 Searching

Search first checks whether the current node is null. If so, the value is absent. If the node's value equals the target, the function returns true. Otherwise, it recursively searches the left subtree for a smaller target or the right subtree for a larger target. The ordering rule eliminates the need to examine the opposite branch.

5.3 Deletion

Deletion is the most structurally involved operation because removing a node may require reconnecting its children. If the target has no left child, its right subtree can take its place. If it has no right child, its left subtree can take its place. If it has two children, the function finds the minimum value in the right subtree, copies that value into the target node, and deletes the successor from its old location. This preserves the ordering rule.

5.4 Depth-first Traversals

Inorder: left subtree, node, right subtree. This is useful for sorted output.
Preorder: node, left subtree, right subtree. This visits a parent before its descendants.
Postorder: left subtree, right subtree, node. This visits descendants before the parent and is also useful when freeing tree nodes.

5.5 Minimum and Maximum

The minimum value is stored in the leftmost node because every leftward step moves to a smaller value. Similarly, the maximum is stored in the rightmost node. Both functions return a null pointer for an empty tree, allowing the menu to show a clear message instead of dereferencing an invalid pointer.

5.6 Input Validation and Memory Cleanup

The input helper checks whether integer extraction succeeded. If it fails, the program clears the stream's error state and ignores the invalid characters remaining on the line. When the user exits, `destroyTree` performs a postorder-style recursive cleanup: it destroys the left and right subtrees before deleting the current node. This releases memory allocated for the tree.

6. TESTING STRATEGY AND RESULTS

Testing should cover ordinary operations as well as boundary cases. A successful demonstration does not only show that values can be inserted; it also checks that the tree remains valid after deletion, that traversal orders are correct, and that empty or invalid inputs do not cause confusing behavior.

6.1 Representative Test Cases

Test	Action	Expected result
TC-01	Insert 50 into an empty tree	50 becomes the root.
TC-02	Insert 30 and 70	30 is left of 50; 70 is right of 50.
TC-03	Insert duplicate 50	Tree remains unchanged; duplicate message appears.
TC-04	Search for 40 when present	Program reports that 40 is found.
TC-05	Search for absent 99	Program reports that 99 is not found.
TC-06	Run inorder on sample tree	Values are printed in ascending order.
TC-07	Run level-order traversal	Nodes are printed one level at a time.
TC-08	Find min and max	For sample tree, minimum is 20 and maximum is 80.
TC-09	Run operations on empty tree	Clear empty-tree message or not-found result.
TC-10	Enter letters for menu choice	Input error is reported and menu can continue.
TC-11	Exit after inserting nodes	All allocated nodes are destroyed before exit.

6.2 Sample Insertion Sequence

Insert the values 50, 30, 70, 20, 40, 60, and 80 in that order. The resulting tree is balanced for this example and has 50 as the root. Expected traversal sequences are shown below.

```
Inorder:      20 30 40 50 60 70 80
Preorder:     50 30 20 40 70 60 80
Postorder:    20 40 30 60 80 70 50
Level-order:  50 30 70 20 40 60 80
```

6.3 Deletion Scenarios

A complete practical demonstration should separately delete a leaf, a node with one child, and a node with two children. For the two-child case, deleting 30 from the sample tree replaces it with its inorder successor 40 and removes the old successor node. The inorder result becomes 20 40 50 60 70 80.

7. COMPLEXITY ANALYSIS

The cost of the core BST operations depends on the height of the tree. Let n represent the number of nodes, h the height of the tree, and w the maximum number of nodes present at any level. In a well-shaped tree, h is proportional to log n. In a skewed tree, h may approach n.

Operation	Time complexity	Auxiliary space	Reason
Insert	O(h), worst O(n)	O(h)	Follows one root-to-leaf path recursively.
Search	O(h), worst O(n)	O(h)	Chooses one subtree at each comparison.
Delete	O(h), worst O(n)	O(h)	Searches for target and possibly successor.
Find min/max	O(h), worst O(n)	O(1)	Follows leftmost or rightmost links iteratively.
Depth-first traversal	O(n)	O(h)	Every node is visited once.
Level-order	O(n)	O(w)	Each node is enqueued and dequeued once.
Display / cleanup	O(n)	O(h)	Processes each node once recursively.

7.1 Balanced-Tree Behavior

If the tree is approximately balanced, its height is O(log n). Insert, search, and delete therefore take approximately logarithmic time because each comparison removes one level from the remaining search path. The traversal operations still take O(n), since they must visit every node to print every value.

7.2 Worst-case Behavior

The BST does not rebalance itself. If values are inserted in sorted order, the tree can become a chain. The height becomes O(n), so insert, search, delete, and minimum/maximum lookup can require O(n) time. A self-balancing tree such as AVL or Red-Black Tree can address this limitation but requires additional rotations and balancing logic.

7.3 Space Usage

Recursive functions use call-stack space proportional to tree height. Level-order traversal uses a queue whose size depends on the maximum width of the tree. The nodes themselves require O(n) memory while the tree exists. `destroyTree` releases those nodes during exit.

8. LIMITATIONS, FUTURE SCOPE AND CONCLUSION

8.1 Current Limitations

The implementation stores integer values only and keeps the tree in memory for one run. It does not support persistence, graphical visualization, balancing, or concurrent access. Since it is a conventional BST, its performance depends on the order in which values are inserted. The input validation handles non-integer input, but the application is still a compact educational console program rather than a production data service.

8.2 Future Scope

- Add AVL or Red-Black Tree balancing to keep height logarithmic.
- Add node count, leaf count, tree height, and count-of-levels operations.
- Add file-based save/load functionality so values persist between runs.
- Create unit tests for all deletion cases and invalid-input paths.
- Add a graphical tree view or print a more compact sideways tree.
- Generalize the node type with templates to support other comparable data types.

8.3 Conclusion

This project demonstrates how a Binary Search Tree organizes values and supports dynamic insertion, searching, deletion, and traversal. The original six menu operations are retained, while the improved version adds breadth-first level-order traversal, minimum and maximum lookup, an indented tree display, clearer empty-tree and duplicate messages, input-stream recovery, and explicit memory cleanup. These changes make the program more informative for learning and easier to demonstrate in a practical examination.

The implementation also illustrates the importance of algorithm analysis. Operations on a BST depend on its height, so a balanced shape can provide fast search paths while a skewed shape may degrade to linear time. The program is therefore a useful foundation for studying more advanced tree structures and balancing techniques.

APPENDIX A — BUILD COMMAND

```
g++ -std=c++11 -Wall -Wextra -pedantic main.cpp -o bst
./bst
```

APPENDIX B — DOCUMENTATION NOTE

The companion Output.pdf.pdf contains terminal-style sample output generated for documentation. It is styled to resemble a console session, but it is not presented as an authentic screenshot captured from a physical terminal. The results shown correspond to the example insertion sequence described in this report.